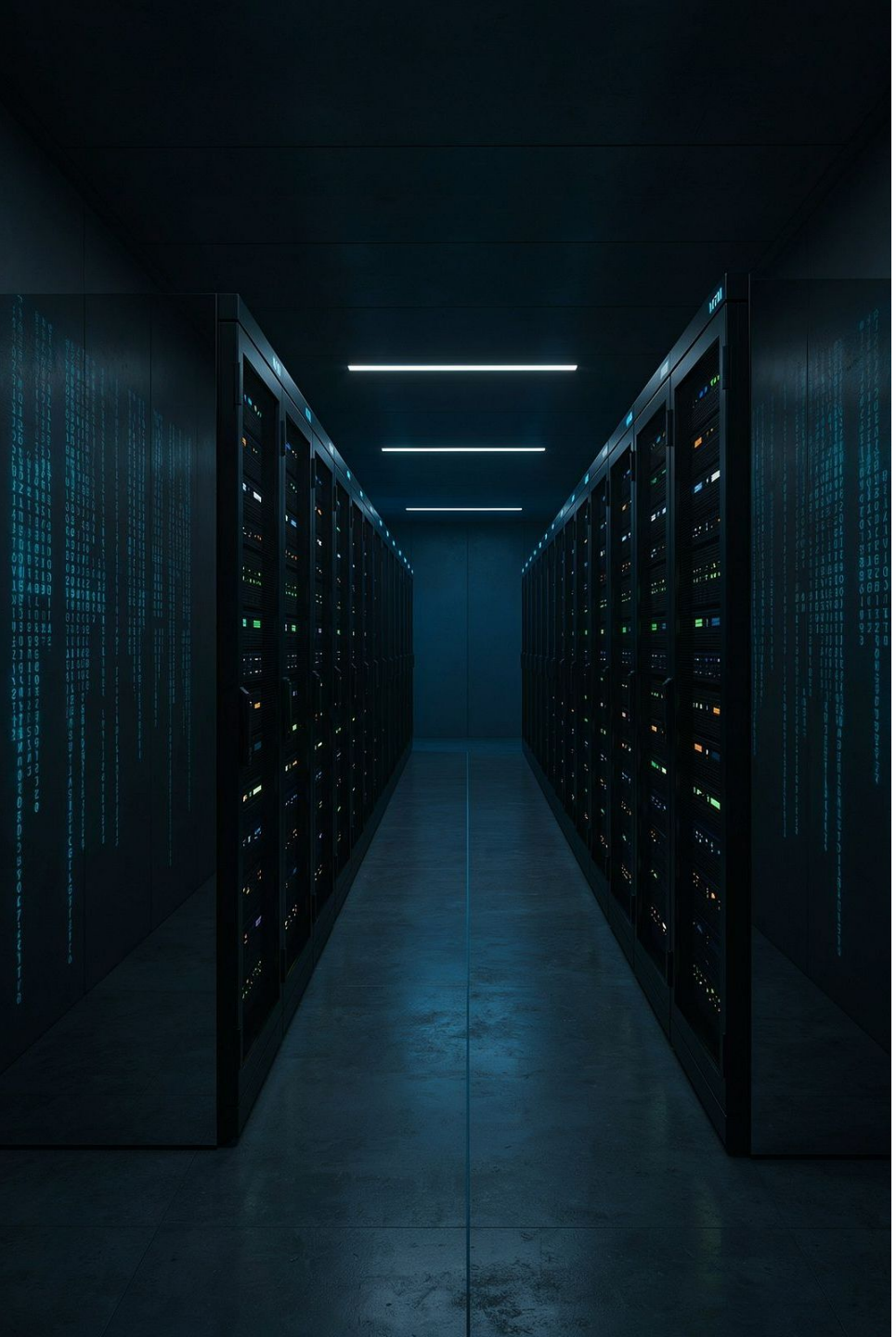


AI-Generated IaC: Power, Pitfalls & Policy Gates

AI coding tools can spin up cloud infrastructure in seconds — but who's checking the blueprints? This presentation walks through why **mandatory plan reviews, OPA/AWS Config gates, and smart developer guardrails** are the three pillars of secure, AI-assisted Infrastructure as Code.

The Problem

AI Writes Fast. Mistakes Spread Faster.

The Promise

A developer asks Amazon Q or GitHub Copilot to generate a Terraform module for an S3 bucket. In 10 seconds, they have 80 lines of production-ready code. Productivity skyrockets. Releases accelerate.

The Hidden Risk

That same AI confidently generates a bucket with **public ACLs enabled**, no encryption, and no access logging — patterns it learned from older open-source repos. Without a human review step, that config ships to production. One misconfigured bucket can expose millions of customer records.

⚠ AI does not know your security posture. It knows patterns from the internet.



The Golden Rule: Mandatory Plan Reviews

Just as no architect would build a skyscraper from unapproved blueprints, no AI-generated IaC should reach production without a human-reviewed **Terraform plan** or **CloudFormation change set**. The plan is your diff — it shows exactly what will be created, changed, or destroyed.

What a Mandatory Plan Review Looks Like

01

AI Generates the Code

Developer prompts Amazon Q Developer to scaffold a VPC, EC2 instances, and IAM roles for a new microservice.

02

Run `terraform plan`

The CI pipeline automatically runs a plan and outputs a human-readable diff. Nothing is applied yet — this is just the preview.

03

Security & Peer Review

A senior engineer (or security team) reviews the plan output in the pull request. Red flags like open security groups or overly permissive IAM policies are caught here.

04

Approved → Apply

Only after explicit approval does the pipeline run `terraform apply`. No approval = no deployment. Full stop.



Chapter 2

Policy Gates: Your Automated Safety Net

Even the best human reviewers miss things. Policy-as-code tools provide a **second layer of enforcement** that never gets tired, never skips a step, and applies the same rules every single time — at scale.

OPA & AWS Config: Real-World Guardrails

Open Policy Agent (OPA)

OPA is a general-purpose policy engine you embed in your CI/CD pipeline. You write rules in a language called **Rego**. For example: *"Any S3 bucket must have server-side encryption enabled."* If AI-generated Terraform violates this rule, the pipeline **fails before anything is deployed**. Think of OPA as the bouncer checking IDs before anyone enters the club.

AWS Config Rules

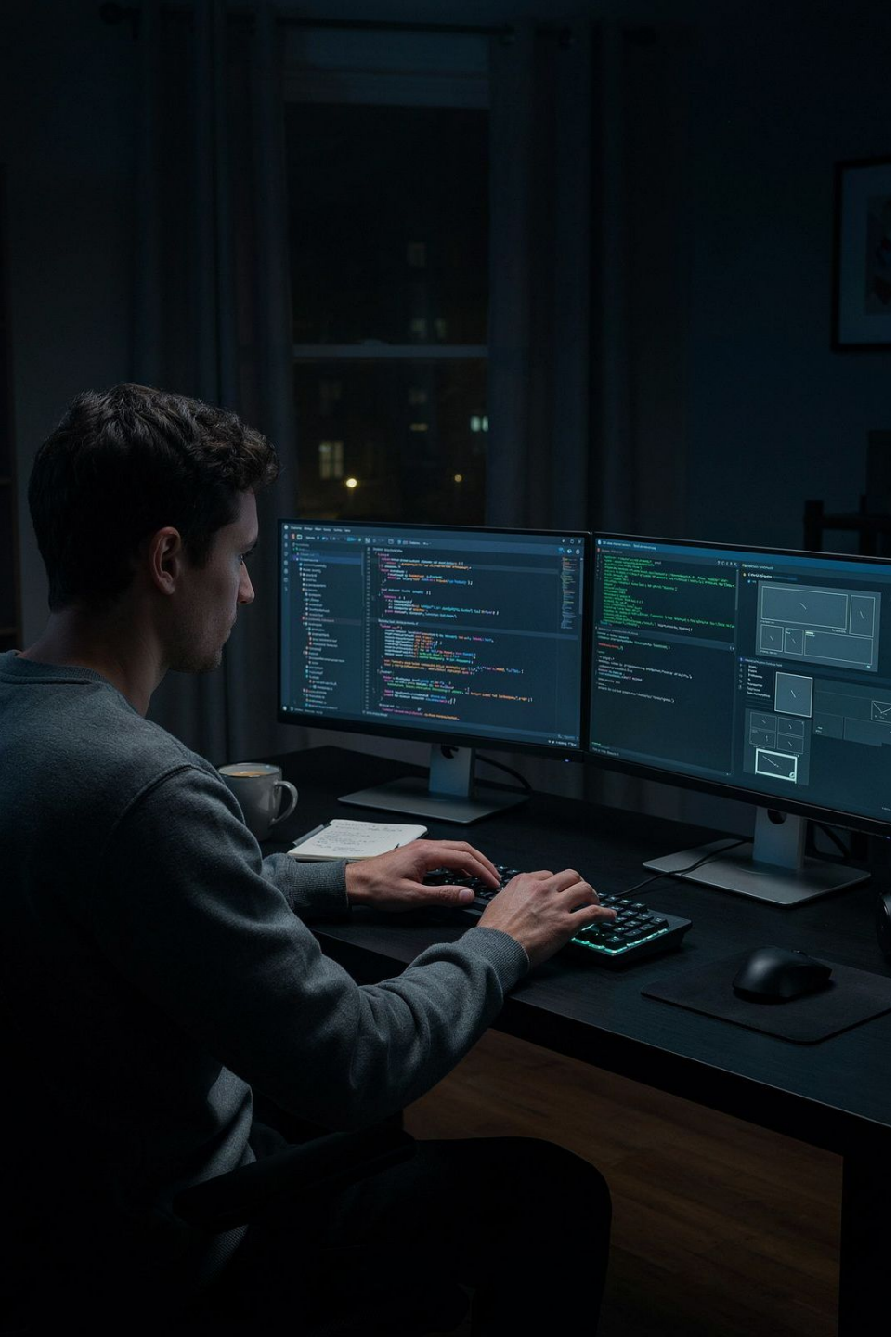
AWS Config continuously monitors your live cloud environment against rules you define. Even if something slips through — a manual change, a legacy resource — AWS Config detects the drift and alerts your team. Common managed rules: **s3-bucket-public-read-prohibited**, **iam-no-inline-policy**, **restricted-ssh**. It's the smoke detector after the fire-prevention system.

OPA in Action: A Simple Example

Imagine AI generates Terraform that opens SSH port 22 to `0.0.0.0/0` (the entire internet). Here is how OPA stops it cold:

```
# Rego policy: deny open SSH to the world
deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_security_group_rule"
  resource.change.after.cidr_blocks[_] == "0.0.0.0/0"
  resource.change.after.from_port <= 22
  resource.change.after.to_port >= 22
  msg := "SSH open to 0.0.0.0/0 is not allowed"
}
```

The pipeline evaluates this policy against the Terraform plan JSON. If the rule fires, the build fails with a clear message — **no human review required for this specific check**. The policy runs every time, automatically.



Chapter 3

Amazon Q Developer Extensions: Guardrails at the Source

The cheapest place to fix a security problem is **before the code is even written**. Amazon Q Developer extensions bring policy awareness directly into the IDE — flagging risky patterns as the AI suggests them, not after they reach a pipeline.

What Amazon Q Developer Extensions Can Do



Security Scans on Generation

Q Developer can run **built-in security scans** on AI-suggested code blocks, catching issues like hardcoded credentials, vulnerable dependencies, or insecure API calls before the developer even saves the file.



Custom Policy Prompts

Teams can configure **custom instructions** that guide Q Developer's suggestions — for example: "Always include S3 bucket versioning" or "Never use wildcard IAM actions." The AI learns your rules and applies them proactively.



CI/CD Workflow Integration

Q Developer integrates with CodePipeline and GitHub Actions, surfacing policy violations as **inline PR comments** with suggested fixes — turning security feedback into a conversational, developer-friendly experience.

The Complete Defense-in-Depth Story

No single layer is enough. The real-world answer is all three working together:

AI Generates IaC

Speed and productivity gains — but treat output as a *first draft*, not finished code

Mandatory Plan Review

Human eyes on the diff in every pull request — no exceptions, no bypasses

OPA / AWS Config Gates

Automated policy enforcement at pipeline and runtime — consistent, tireless, scalable

Q Developer Extensions

Shift security left — catch problems at the moment of code generation, not after deployment

The goal is not to slow down AI-assisted development — it is to make it **sustainably fast** by building trust through verified, policy-compliant infrastructure every time.